CLINICAL PROGRAMMING BOOTCAMP · MODULE 16

Basic Data Structure

One row per subject per parameter per visit

The shape behind nearly every efficacy dataset you will ever build.

Hands-on: Notebook 16 (SAS) — Build ADVS

MODULE GOALS

By the end of this module you can…

1. **Build the BDS skeleton** PARAMCD / PARAM / PARAMN, AVAL, AVISIT — and say what each is for.
2. **Distinguish AVISIT from VISIT** and explain why a study keeps both.
3. **Add a derived parameter** and know why DTYPE and SRCDOM stay empty on it.
4. **Derive BASE, CHG and PCHG** including the two rules that quietly bias results.
5. **Use ANL01FL** to exclude records without deleting them.

THE SHAPE

SDTM VS is already tall — BDS re-labels it for analysis

One subject, one parameter, three visits — and the columns a table needs, already on the row.

PARAMCD	AVISIT	ADY	AVAL	ABLFL	BASE	CHG	ANL01FL
SYSBP	Screening	-10	122		120		
SYSBP	Baseline	1	120	**Y**	120		Y
SYSBP	Week 4	28	118		120	-2	Y

Screening was collected and is KEPT — it is simply not flagged for analysis.

Three variables carry the parameter — and they must be one-to-one

PARAMCD the short key you subset on
PARAM what prints as the row label — so it carries the UNIT
PARAMN what controls print ORDER — without it, Diastolic sorts above Systolic

VISITS

AVISIT is an analysis visit, not a collected one

What the CRF collected

VISIT	VISITNUM
SCREENING	1
BASELINE	2
WEEK 4	4

What the analysis reports

AVISIT	AVISITN
Screening	0
Baseline	1
Week 4	4

Deliberately different numbers. Both are kept.

AVISITN drives sort order and column order in every table. It is chosen to suit the table, not the CRF.

Analysis visits can also WINDOW (a Week 4 visit collected on day 26 or day 31 still reports as Week 4), MERGE, or DROP collected visits entirely. Keeping both columns means a reviewer can always see what was collected AND how it was analysed — which is the whole traceability argument in miniature.

DERIVED PARAMETERS

BMI exists in no SDTM domain

A NEW PARAMCD

```
PARAMCD = 'BMI'
AVAL    = round( weight / (HEIGHTBL/100)**2 , 0.01 )

the visit's weight  ·  the subject's BASELINE height  ·  height is measured once
```

`DTYPE stays EMPTY`

DTYPE identifies a derived RECORD WITHIN a parameter — LOCF, a replicate average. A whole new parameter is not that.

Populate it and a reviewer concludes these rows were imputed. They were not.

`SRCDOM / SRCSEQ stay EMPTY`

BMI comes from a weight record AND a height record AND ADSL. There is no single source row to name.

Naming just the weight row would be a false trail. The blank is information.

Derived PARAMETER (new PARAMCD, DTYPE null) ≠ derived RECORD (existing PARAMCD, DTYPE populated).

TWO RULES

Both quietly bias results if you get them wrong

1 · CHG at baseline is BLANK, not zero

A baseline record's change from baseline is UNDEFINED. Writing 0 puts it in the mean-change column.

`WEIGHT, mean change from baseline:` **`-0.3375 kg`** correct **`-0.1688 kg`** with zeros

Zeros add nothing to the numerator and increase the denominator — so the bias is ALWAYS toward the null. It never invents a signal; it only ever hides one.

2 · Rounding is specified, not incidental

`69.8 - 70.2 = -0.4000000000000057` in SAS and in Python alike

The spec names the precision — ROUND(CHG, 0.0001) — and both implementations apply it, so PROC COMPARE is exactly clean rather than nearly clean.

Tolerated numerical noise trains people to ignore PROC COMPARE — and then the run where a difference is 0.4 gets waved through too.

ANL01FL

Exclude records without deleting them

Delete 48 Screening rows. Same count, two orders, two completely different outcomes.

AFTER deriving ABLFL

```
rows        152 → 104
baselines    56 →  56
HEIGHT        8 →   8
```

Harmless — and pointless. ANL01FL already did this, without discarding data.

BEFORE deriving ABLFL

```
rows        152 → 104
baselines    56 →  48
HEIGHT        8 →   0
```

HEIGHT destroyed completely. No error. It looks like a study that never measured height.

Why the order decides

The baseline rule is a statement about TIME, and it can only be evaluated against the full history. Filtering first throws away the evidence the rule needs — and the rule then reports, correctly and silently, that there is no baseline.

Flagging is reversible and self-documenting. Filtering is destructive and silent. That is why ADaM is full of flags rather than filtered datasets.

WHAT'S NEXT

152 rows, and one of them has no change

1 **Step 1 · Notebook 16** Build ADVS: 128 observed rows plus 24 derived BMI records.

2 **Step 2 · Check the baselines** PROC FREQ of PARAMCD × AVISIT where ABLFL='Y'. HEIGHT must show Screening.

3 **Step 3 · Exercise 4** Delete the Screening rows twice, in two different orders. Watch HEIGHT vanish.

Tomorrow: the same skeleton, three more columns — and only half the study has any data at all.